NoteA new Javascript/Typescript SDK, @google/genai (github), is currently available in a experimental preview launch - designed to work with Gemini 2.0 features. and support both the Gemini API and the Vertex API.

Version latest

Vertex AI SDK for Node.js quickstart

The Vertex AI SDK for Node.js lets you use the Vertex AI Gemini API to build AI-powered features and applications. Both TypeScript and JavaScript are supported. The sample code in this document is written in JavaScript only.

For detailed samples using the Vertex AI Node.js SDK, see the samples repository (https://github.com/GoogleCloudPlatform/nodejs-docs-samples/tree/main/generative-ai/snippets) on GitHub.

For the latest list of available Gemini models on Vertex AI, see the Model information (https://cloud.google.com/vertex-ai/docs/generative-ai/learn/models#gemini-models) page in Vertex AI documentation.

Before you begin

1. Make sure your node.js version is 18 or above.
2. Select (https://console.cloud.google.com/project) or create (https://cloud.google.com/resource-manager/docs/creating-managing-projects#creating_a_project)
   a Google Cloud project.
3. Enable billing for your project (https://cloud.google.com/billing/docs/how-to/modify-project).
4. Enable the Vertex AI API (https://console.cloud.google.com/flows/enableapi?apiid=aiplatform.googleapis.com).

5. Install the gcloud CLI (https://cloud.google.com/sdk/docs/install).
6. Initialize the gcloud CLI (https://cloud.google.com/sdk/docs/initializing).
7. Create local authentication credentials for your user account:

   ```
   gcloud auth application-default login
   ```

   A list of accepted authentication options are listed in GoogleAuthOptions (https://github.com/googleapis/google-auth-library-nodejs/blob/3ae120d0a45c95e36c59c9ac8286483938781f30/src/auth/googleauth.ts#L87) interface of google-auth-library-node.js GitHub repo.
8. Official documentation is available in the Vertex AI SDK Overview (https://cloud.google.com/vertex-ai/generative-ai/docs/reference/nodejs/latest/overview) page. From here, a complete list of documentation on classes, interfaces, and enums are available.

Install the SDK

Install the Vertex AI SDK for Node.js by running the following command:

```
npm install @google-cloud/vertexai
```

Initialize the `VertexAI` class

To use the Vertex AI SDK for Node.js, create an instance of `VertexAI` by passing it your Google Cloud project ID and location. Then create an instance of the GenerativeModel class using the VertexAI class methods.

```
const {
  FunctionDeclarationSchemaType,
  HarmBlockThreshold,
  HarmCategory,
  VertexAI
```

```
} = require('@google-cloud/vertexai (https://docs.cloud.google.com/nodejs/docs/reference/

const project = 'your-cloud-project';
const location = 'us-central1';
const textModel =  'gemini-1.5-flash';
const visionModel = 'gemini-1.5-flash';

const vertexAI = new VertexAI (https://docs.cloud.google.com/nodejs/docs/reference/vertexai/

// Instantiate Gemini models
const generativeModel = vertexAI.getGenerativeModel (https://docs.cloud.google.com/no
    model: textModel,
    // The following parameters are optional
    // They can also be passed to individual content generation requests
    safetySettings: [{category: HarmCategory (https://docs.cloud.google.com/nodejs/docs
    generationConfig: {maxOutputTokens: 256},
    systemInstruction: {
      role: 'system',
      parts: [{"text": `For example, you are a helpful customer service agent
    },
});

const generativeVisionModel = vertexAI.getGenerativeModel (https://docs.cloud.google.
    model: visionModel,
});

const generativeModelPreview = vertexAI.preview (https://docs.cloud.google.com/nodejs/
    model: textModel,
});
```

Send text prompt requests

You can send text prompt requests by using **`generateContentStream`** for streamed responses, or **`generateContent`** for nonstreamed responses.

Get streamed text responses

The response is returned in chunks as it's being generated to reduce the perception of latency to a human reader.

```
async function streamGenerateContent() {
  const request = {
    contents: [{role: 'user', parts: [{text: 'How are you doing today?'}]}],
  };
  const streamingResult = await generativeModel.generateContentStream(request
  for await (const item of streamingResult.stream) {
    console.log('stream chunk: ', JSON.stringify(item));
  }
  const aggregatedResponse = await streamingResult.response;
  console.log('aggregated response: ', JSON.stringify(aggregatedResponse));
};

streamGenerateContent();
```

Get nonstreamed text responses

The response is returned all at once.

```
async function generateContent() {
  const request = {
    contents: [{role: 'user', parts: [{text: 'How are you doing today?'}]}],
  };
  const result = await generativeModel.generateContent(request);
  const response = result.response;
  console.log('Response: ', JSON.stringify(response));
};

generateContent();
```

Send multiturn chat requests

Chat requests use previous messages as context when responding to new prompts. To send multiturn chat requests, use `sendMessageStream` for streamed responses, or `sendMessage` for nonstreamed responses.

Get streamed chat responses

The response is returned in chunks as it's being generated to reduce the perception of latency to a human reader.

```
async function streamChat() {
  const chat = generativeModel.startChat();
  const chatInput = "How can I learn more about Node.js?";
  const result = await chat.sendMessageStream(chatInput);
  for await (const item of result.stream) {
      console.log("Stream chunk: ", item.candidates[0].content.parts[0].text)
  }
  const aggregatedResponse = await result.response;
  console.log('Aggregated response: ', JSON.stringify(aggregatedResponse));
}

streamChat();
```

Get nonstreamed chat responses

The response is returned all at once.

```
async function sendChat() {
  const chat = generativeModel.startChat();
  const chatInput = "How can I learn more about Node.js?";
  const result = await chat.sendMessage(chatInput);
  const response = result.response;
  console.log('response: ', JSON.stringify(response));
}

sendChat();
```

Include images or videos in your prompt request

Prompt requests can include either an image or video in addition to text. For more information, see Send multimodal prompt requests (https://cloud.google.com/vertex-ai/generative-ai/docs/multimodal/send-multimodal-prompts) in the Vertex AI documentation.

Include an image

You can include images in the prompt either by specifying the Cloud Storage URI where the image is located or by including a base64 encoding of the image.

Specify a Cloud Storage URI of the image

You can specify the Cloud Storage URI of the image in **`fileUri`**.

```
async function multiPartContent() {
    const filePart = {fileData: {fileUri: "gs://generativeai-downloads/images/
    const textPart = {text: 'What is this picture about?'};
    const request = {
        contents: [{role: 'user', parts: [textPart, filePart]}],
      };
    const streamingResult = await generativeVisionModel.generateContentStream
    for await (const item of streamingResult.stream) {
      console.log('stream chunk: ', JSON.stringify(item));
    }
    const aggregatedResponse = await streamingResult.response;
    console.log(aggregatedResponse.candidates[0].content);
}

multiPartContent();
```

Specify a base64 image encoding string

You can specify the base64 image encoding string in **`data`**.

```
async function multiPartContentImageString() {
    // Replace this with your own base64 image string
    const base64Image = 'iVBORw0KGgoAAAANSUhEUgAAAAEAAAABCAYAAAAfFcSJAAAADUlE
    const filePart = {inline_data: {data: base64Image, mimeType: 'image/jpeg'
    const textPart = {text: 'What is this picture about?'};
    const request = {
        contents: [{role: 'user', parts: [textPart, filePart]}],
      };
    const streamingResult = await generativeVisionModel.generateContentStream
    const contentResponse = await streamingResult.response;
    console.log(contentResponse.candidates[0].content.parts[0].text);
}
```

```
multiPartContentImageString();
```

Include a video

You can include videos in the prompt by specifying the Cloud Storage URI where the video is located in `fileUri`.

```
async function multiPartContentVideo() {
    const filePart = {fileData: {fileUri: 'gs://cloud-samples-data/video/anima
    const textPart = {text: 'What is in the video?'};
    const request = {
        contents: [{role: 'user', parts: [textPart, filePart]}],
      };
    const streamingResult = await generativeVisionModel.generateContentStream
    for await (const item of streamingResult.stream) {
      console.log('stream chunk: ', JSON.stringify(item));
    }
    const aggregatedResponse = await streamingResult.response;
    console.log(aggregatedResponse.candidates[0].content);
}

multiPartContentVideo();
```

Function calling

The Vertex AI SDK for Node.js supports function calling (https://cloud.google.com/vertex-ai/docs/generative-ai/multimodal/function-calling) in the `sendMessage`, `sendMessageStream`, `generateContent`, and `generateContentStream` methods. We recommend using it through the chat methods (`sendMessage` or `sendMessageStream`) but have included examples of both approaches below.

Declare a function

The following examples show you how to declare a function.

```
const functionDeclarations = [
  {
    functionDeclarations: [
      {
        name: "get_current_weather",
        description: 'get weather in a given location',
        parameters: {
          type: FunctionDeclarationSchemaType.OBJECT,
          properties: {
            location: {type: FunctionDeclarationSchemaType.STRING},
            unit: {
              type: FunctionDeclarationSchemaType.STRING,
              enum: ['celsius', 'fahrenheit'],
            },
          },
          required: ['location'],
        },
      },
    ],
  },
];

const functionResponseParts = [
  {
    functionResponse: {
      name: "get_current_weather",
      response:
          {name: "get_current_weather", content: {weather: "super nice"}},
    },
  },
];
```

Function calling using `sendMessageStream`

After the function is declared, you can pass it to the model in the `tools` parameter of the prompt request.

```
async function functionCallingChat() {
  // Create a chat session and pass your function declarations
  const chat = generativeModel.startChat({
    tools: functionDeclarations,
  });
```

```
  const chatInput1 = 'What is the weather in Boston?';

  // This should include a functionCall response from the model
  const streamingResult1 = await chat.sendMessageStream(chatInput1);
  for await (const item of streamingResult1.stream) {
    console.log(item.candidates[0]);
  }
  const response1 = await streamingResult1.response;
  console.log("first aggregated response: ", JSON.stringify(response1));

  // Send a follow up message with a FunctionResponse
  const streamingResult2 = await chat.sendMessageStream(functionResponseParts
  for await (const item of streamingResult2.stream) {
    console.log(item.candidates[0]);
  }

  // This should include a text response from the model using the response co
  // provided above
  const response2 = await streamingResult2.response;
  console.log("second aggregated response: ", JSON.stringify(response2));
}

functionCallingChat();
```

Function calling using `generateContentStream`

```
async function functionCallingGenerateContentStream() {
  const request = {
    contents: [
      {role: 'user', parts: [{text: 'What is the weather in Boston?'}]},
      {role: 'model', parts: [{functionCall: {name: 'get_current_weather', ar
      {role: 'user', parts: functionResponseParts}
    ],
    tools: functionDeclarations,
  };
  const streamingResult =
      await generativeModel.generateContentStream(request);
  for await (const item of streamingResult.stream) {
    console.log(item.candidates[0]);
  }
}
```

```
functionCallingGenerateContentStream();
```

Counting tokens

```
async function countTokens() {
  const request = {
      contents: [{role: 'user', parts: [{text: 'How are you doing today?'}]}]
    };
  const response = await generativeModel.countTokens(request);
  console.log('count tokens response: ', JSON.stringify(response));
}

countTokens();
```

Grounding (Preview)

Grounding is preview only feature.

Grounding lets you connect model output to verifiable sources of information to reduce hallucination. You can specify Google Search or Vertex AI search as the data source for grounding.

Grounding using Google Search (Preview)

```
async function generateContentWithGoogleSearchGrounding() {
  const generativeModelPreview = vertexAI.preview.getGenerativeModel({
    model: textModel,
    // The following parameters are optional
    // They can also be passed to individual content generation requests
    safetySettings: [{category: HarmCategory.HARM_CATEGORY_DANGEROUS_CONTENT,
    generationConfig: {maxOutputTokens: 256},
  });

  const googleSearchRetrievalTool = {
    googleSearchRetrieval: {
```

```
      disableAttribution: false,
    },
  };
  const result = await generativeModelPreview.generateContent({
    contents: [{role: 'user', parts: [{text: 'Why is the sky blue?'}]}],
    tools: [googleSearchRetrievalTool],
  })
  const response = result.response;
  const groundingMetadata = response.candidates[0].groundingMetadata;
  console.log("GroundingMetadata is: ", JSON.stringify(groundingMetadata));
}
generateContentWithGoogleSearchGrounding();
```

Grounding using Vertex AI Search (Preview)

```
async function generateContentWithVertexAISearchGrounding() {
  const generativeModelPreview = vertexAI.preview.getGenerativeModel({
    model: textModel,
    // The following parameters are optional
    // They can also be passed to individual content generation requests
    safetySettings: [{category: HarmCategory.HARM_CATEGORY_DANGEROUS_CONTENT,
    generationConfig: {maxOutputTokens: 256},
  });

  const vertexAIRetrievalTool = {
    retrieval: {
      vertexAiSearch: {
        datastore: 'projects/.../locations/.../collections/.../dataStores/...
      },
      disableAttribution: false,
    },
  };
  const result = await generativeModelPreview.generateContent({
    contents: [{role: 'user', parts: [{text: 'Why is the sky blue?'}]}],
    tools: [vertexAIRetrievalTool],
  })
  const response = result.response;
  const groundingMetadata = response.candidates[0].groundingMetadata;
  console.log("Grounding metadata is: ", JSON.stringify(groundingMetadata));
}
generateContentWithVertexAISearchGrounding();
```

System Instruction

You can include an optional system instruction when instantiating a generative model to provide additional context to the model.

The system instruction can also be passed to individual text prompt requests.

Include system instruction in generative model instantiation

```
const generativeModel = vertexAI.getGenerativeModel({
    model: textModel,
    // The following parameter is optional.
    systemInstruction: {
      role: 'system',
      parts: [{"text": `For example, you are a helpful customer service agent
    },
});
```

Include system instruction in text prompt request

```
async function generateContent() {
  const request = {
    contents: [{role: 'user', parts: [{text: 'How are you doing today?'}]}],
    systemInstruction: { role: 'system', parts: [{ text: `For example, you are
  };
  const result = await generativeModel.generateContent(request);
  const response = result.response;
  console.log('Response: ', JSON.stringify(response));
};

generateContent();
```

FAQ

What if I want to specify authentication options instead of using default options?

Step1: Find a list of accepted authentication options in GoogleAuthOptions (https://github.com/googleapis/google-auth-library-nodejs/blob/3ae120d0a45c95e36c59c9ac8286483938781f30/src/auth/googleauth.ts#L87) interface of google-auth-library-node.js GitHub repo.

Step2: Instantiate the `VertexAI` class by passing in the `GoogleAuthOptions` interface as follows:

```
const { VertexAI } = require('@google-cloud/vertexai (https://docs.cloud.google.com/n
const { GoogleAuthOptions } = require('google-auth-library (https://docs.cloud.googl
const vertexAI = new VertexAI (https://docs.cloud.google.com/nodejs/docs/reference/vertexai/
  {
    googleAuthOptions: {
      // your GoogleAuthOptions interface
    }
  }
)
```

License

The contents of this repository are licensed under the Apache License, version 2.0 (http://www.apache.org/licenses/LICENSE-2.0).

Except as otherwise noted, the content of this page is licensed under the Creative Commons Attribution 4.0 License (https://creativecommons.org/licenses/by/4.0/), and code samples are licensed under the Apache 2.0 License (https://www.apache.org/licenses/LICENSE-2.0). For details, see the Google Developers Site Policies (https://developers.google.com/site-policies). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2025-10-30 UTC.